Accenture Japan OA

Grid and Buildings — Reconstructed Problem Statement & Solution

Problem statement

A city is represented by an **N × N grid of parcels**, formed by N+1 roads in each of two perpendicular directions. Each parcel is a square. Every group of four adjacent parcels forms a **2 × 2 block**.

There are M buildings. For each building, the input gives coordinates (A_i, B_i) of the parcel containing it. An integer K specifies the maximum number of buildings allowed in any block.

Task: Determine whether every possible 2 × 2 block contains at most K buildings. Print **YES** if the condition holds for all blocks; otherwise, print **NO**.

Input format

```
N M K
A1 B1
A2 B2
...
AM BM
```

Output format

Print YES if every 2 × 2 block contains at most K buildings; otherwise, print NO.

How to check a block

For a block whose top-left parcel is (i, j), add the building counts in its four parcels:

```
S = grid[i][j] + grid[i+1][j]
  + grid[i][j+1] + grid[i+1][j+1]
```

If S > K for any block, the answer is NO immediately. If every block passes the check, the answer is YES.

C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M, K;
    cin >> N >> M >> K;

    vector<vector<int>> grid(N + 1, vector<int>(N + 1, 0));

    for (int i = 0; i < M; i++) {
        int A, B;
        cin >> A >> B;
        grid[A][B]++;
    }

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int buildings = grid[i][j]
                          + grid[i + 1][j]
                          + grid[i][j + 1]
                          + grid[i + 1][j + 1];

            if (buildings > K) {
                cout << "NO\n";
                return 0;
            }
        }
    }

    cout << "YES\n";
```

```
    return 0;
}
```

Complexity

Time	$O(N^2 + M)$: read M buildings, then inspect each 2 × 2 block.
Space	$O(N^2)$: store the building count for each parcel.

Why the sliding-window idea works: every 2 × 2 block is a fixed-size window. Checking its four cells directly is sufficient; no one-dimensional sliding-window machinery is needed.

Assumptions: This document reconstructs the problem from the remembered description; it is not verified original wording. The code assumes parcel coordinates are indexed from 0 through N and that each input coordinate identifies one parcel. Adjust indexing if the original statement uses 1-based parcel coordinates.